15 minutes, then you build

How ScorePlay works, and how to build on it

Jonas Verest, Deployment Lead, ScorePlay

Spurs² Hackathon, UT Austin

ScorePlay

Part 1 · What ScorePlay is

Who we are, one library, one pipeline.

Who we are: the media platform sports organisations run on

ScorePlay stores, tags and distributes every photo and video a sports organisation produces: live, fresh from the match, and from the archive.

01

Ingest

every route in: upload, FTP, live feeds, on-site storage, archives

02

Index

AI tags players and sponsors; everything searchable

03

Produce

clip and log moments, in the platform

04

Distribute

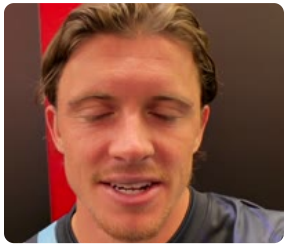
share with sponsors, broadcasters and partners, each with its own rights window

For Sports. By Sports.

One library for everything a club produces



Photo
PC1_9993.JPG, 4204x2803
130001124



Raw video from a phone
IMG_2434.MOV, 2160x3840, 60 fps
130000893



Full match
1920x1080, 6,334 s
130000990
Manchester United vs Spurs | Full Match | 2026-27.mp4



Press conference
1920x1080, 725 s
130001030
Press Conference: De Zerbi speaks to the media ahead of ...



Finished 9:16 edit
1080x1920, 119 s
130000987
9x16_MNU_1_1_TOT_HIGHLIGHTS.mp4



Motion graphic
Slide 1 - Goal.mov, 3400x2160, 20 s
130001115

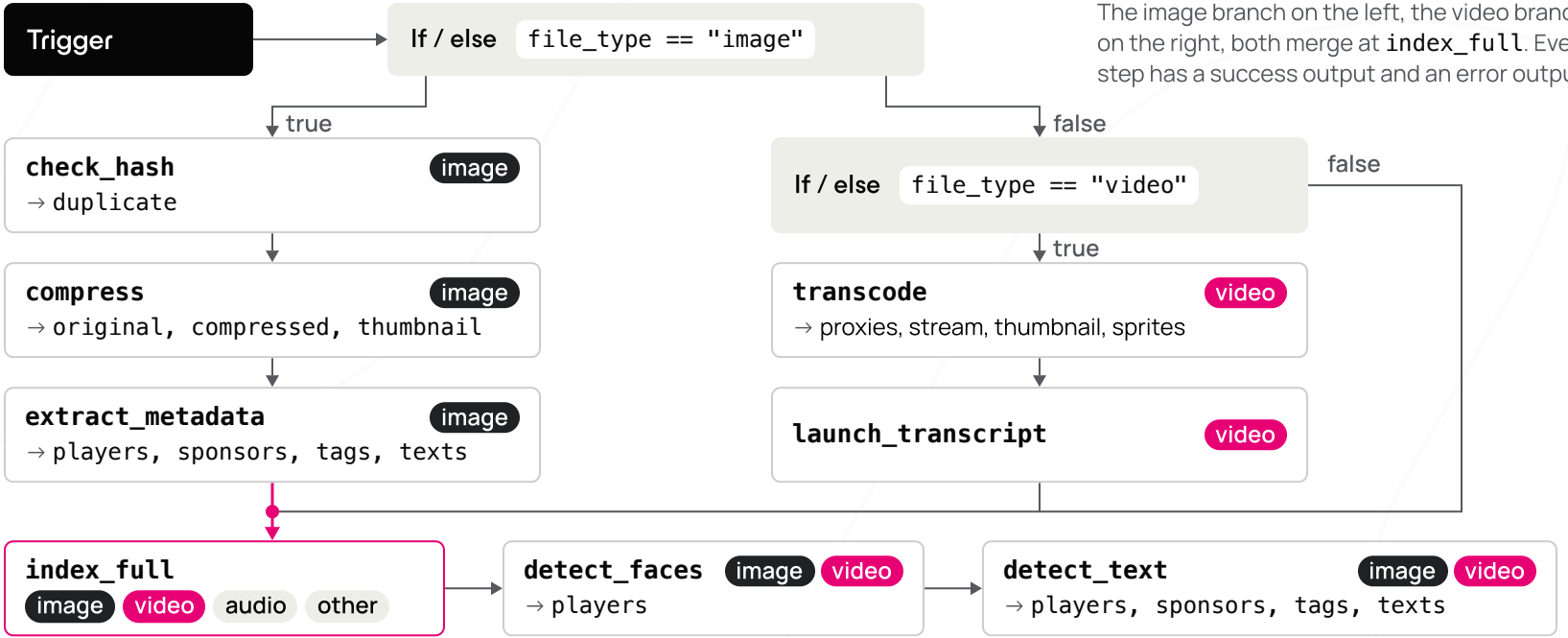
● Files arrive by | Web upload | API upload | Live photos (FTP) | Live video | On-site agent | Archives and provider drops

Your platform: 476 media, 418 photos and 58 videos.

Every upload runs as a workflow

A graph of steps, run on Temporal. Each organisation can customise it, per source. **Fixed this week.**

- **The default workflow**



The image branch on the left, the video branch on the right, both merge at `index_full`. Every step has a success output and an error output.

A workflow is built from steps

- The steps a workflow can use

Image

`check_hash`	checks the file's hash for duplicates
`compress`	makes the compressed and thumbnail files
`extract_metadata`	reads EXIF and XMP metadata into fields and tags
`detect_faces`	identifies players from their faces
`detect_text`	reads text in the frame: names, numbers, boards
`detect_logos`	detects sponsor logos

Video

`transcode`	makes the proxies, the stream and the previews
`ai_reframing`	makes a cropped version
`launch_transcript`	starts the transcript

All media

`index_full`	makes the media searchable
`add_media_tags`	adds set tags
`check_duplicate_name`	checks for a media with the same name
`http_call`	calls an outside service

Each organisation picks and orders its steps, per source. `http_call` plugs in an outside service, for example a club's own enrichment.

Part 2 · How we structure media

Collection, media, version, file, tag, log.



A collection holds media, a media holds versions, a version holds files



`130000990`

Manchester United vs Spurs I Full Match I 2026-27.mp4

1 Collection API object `Event`, field `event_id`

`Premier League` › `Manchester United vs Tottenham Hotspur – Week 6` › `Match`. A media sits in exactly one collection.

2 Media `ID`, `type`

`130000990`, Manchester United vs Spurs I Full Match I 2026-27.mp4, video

Fields on it: `name`, `date`, `captured_time`, `description`, `tc_in`, `tc_out`, `players`, `tags`

3 Version `MediaVersion`

`default`. An edit saved back becomes a second version.

4 Files

original 8.1 GB, 1080p proxy 5.7 GB, 720p proxy 2.6 GB.

Also a thumbnail, an HLS stream and sprite strips.

5 Tags and logs

Tags hang off the media (the full match carries 11 player tags).

Logs sit on the video's timeline: timecoded moments with their own tags, players and comment (none yet).

A collection holds media. A media holds versions, a version holds files. Tags describe a media. A log is a tagged moment on a video. Search reads all of it; the app you build writes all of it back.

A media carries its own facts: description, captured time, date



Photo 130001124

PC1_9993.JPG



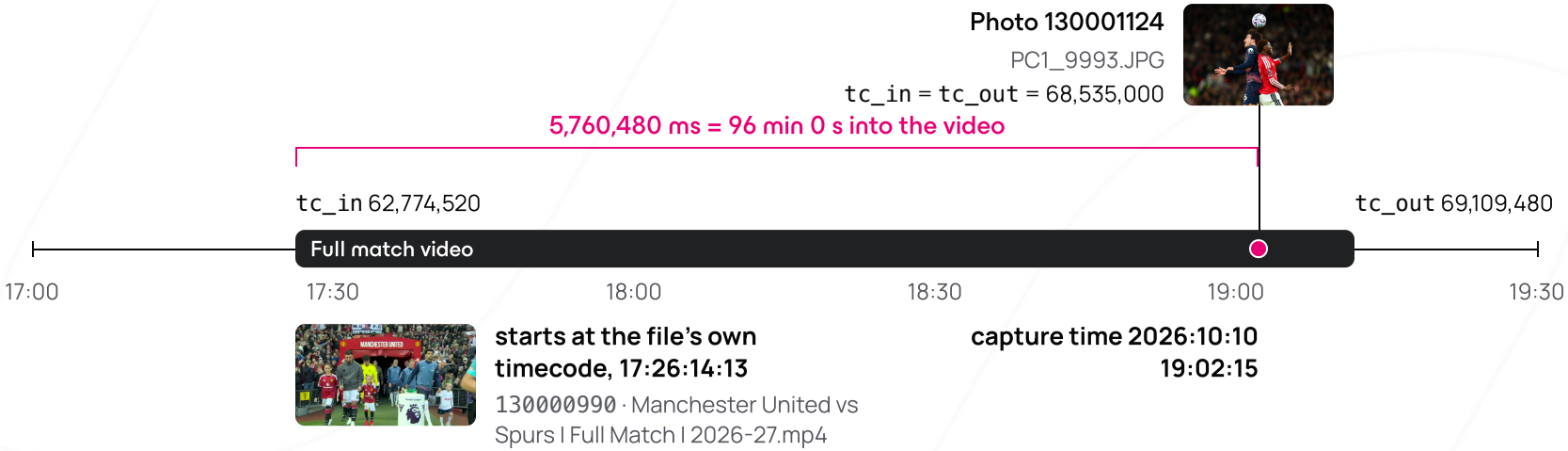
Full match 130000990

Manchester United vs Spurs I Full Match I 2026-27.mp4

`description`	Sandro Tonali of Tottenham Hotspur battles in the air	*empty*
`captured_time`	`2026:10:10 19:02:15`	*empty*
`date`	17 Oct 2026	17 Oct 2026
size	4204x2803	`video`: 1920x1080, 25 fps, 6,334 s

`date` is the library date (here, the day the content was loaded). `description` is searchable. Also on a media: `tc_in`, `tc_out` (next slide), `players`, `tags`.

Photos and videos sit on one clock: time of day in milliseconds



1 A video starts at its file's timecode, or at its creation time if the file has none.

2 A photo sits at its capture time.

3 Both are stored as `tc_in` and `tc_out`: **milliseconds since midnight**.

Only as good as the cameras' clocks: check one frame by eye. Logs written as unix times are placed with the video's start; on these platforms write `start_offset` in seconds instead.

A collection passes its tags down to its media

Premier League	tags: Premier League, 2026/27, Soccer
└ Manchester United vs Tottenham Hotspur - Week 6	+ Away, Old Trafford, Week 6, Manchester United, Tottenham Hotspur, Men's First Team, **Regular Season** ...
└ Pre-Match 11	+ pre-match
└ Match 399	+ Match Day
└ Moments 13	
└ Dressing Room 14	
└ Creative Video 19	
└ Post-Match 5	+ post-match
└ Press Conference 10	+ press conference
└ Highlights 5	+ Highlights

- `Regular Season` is set once, on the match collection, and it is on all 476 media.

Tags describe the whole media, and come from four places



Photo 130001124

PC1_9993.JPG

20 tags, by where they come from

Players	Sandro Tonali
Its collections (15)	Premier League, Week 6, Old Trafford, Manchester United, Tottenham Hotspur, Match Day, 2026/27, Soccer, ...
The file (3)	`Photo` `Landscape` `4204x2803`
Credit	`paul currie`

A tag value is a **tag option**; tag options sit in **tag groups**: sport, season, category, team, credits, format, in-game actions.

On ingest, ScorePlay tags players and sponsors from the frame



● **Player tag: Sandro Tonali**

Photo 130001124 · PC1_9993.JPG

What	Read from	Becomes
Players	faces, matched to the players' headshots; names and numbers on shirts	a player tag
Sponsors	logos; text on boards, shirts, backdrops	a sponsor tag

Every photo +the first two minutes of every video.

Speech is different: the whole video becomes a timed transcript, in English, French and Spanish here.
`GET /media/{id}/transcripts`

On your platform: 322 of 418 photos and 37 of 58 videos carry a player tag. No media carries a sponsor tag.

A tag says who is in it, not when: logs are the moments



130000990

Manchester United vs Spurs I Full Match I 2026-27.mp4

Tags on the whole media: 11 players.

Logs: 0.

0 s

6,334 s

```
GET /media/130000990/logs
{"items": [], "page": 1, "limit": 50}
```

- **Tags** describe the whole media. ScorePlay adds them on ingest.
- **Logs** are timecoded moments: a start, an end, tags, players, a comment. None are created by default: the app you build writes them.

Part 3 · The API, and how to use it this week

Eight calls, in the order of the build loop.



One header, one base URL: your first call is /whoami

```
curl https://media.scoreplay.io/v1/whoami \
  -H "X-ScorePlay-Api-Key: $KEY"

{"user_id": 85146, "user_type": "admin",
 "company": {"name": "spurs2-hack-10", ...},
 "rights": ["can_do_media_download", ...], "scope": {...}}
```

- **Your key:** create it in the platform under Settings › API key. A team admin key has 175 `rights`.
- **A missing right returns `401`**, the same as a bad key.
- **`company_id`** (5167 here) is needed to write logs. It is not in `/whoami`: read it off any media.

Your IDs will differ from the ones on these slides. Same content, different platform.

Find: from a name to its media in a few calls

```
GET /tag/search?query=solanke
→ {"players": [{"ID": 728708, "full_name": "Dominic Ayodele Solanke-Mitchell", ...}],
   "tags": [], "sponsors": []}
POST /event/search {"sub_and_master": true}   → the collections, as a bare array
POST /media/search {"players": [728708], "media_type": "video"}   → count 15
POST /media/search {"players": [728708], "media_type": "photo"}   → count 53
```

Filters you will use: `players`, `tag_options`, `event_id`, `event_ids`, `media_type`, `transcript_query`. Add `any_` for OR, `not_` to exclude.

Search fails quietly: four traps

- Four traps

Trap	What you see	Do this
`event_id` is not recursive	`Match` returns 399 media; with its 3 sub-collections 445; the match root 0	send `event_ids` with every ID
An unknown filter key is ignored	`type: "photo"` returns all 476; `media_type: "photo"` returns 418	test each filter with a fake ID: it must return 0
`tag_options` means AND	a stale ID returns 0 results, no error	check your IDs before you search
Tag search matches substrings	`son` returns Robertson, Maddison, Richarlison…	pick by ID

Fetch the proxy, and keep the work on your machine

- Three names for the same files

Where	Names
`media.files[].details`	`photographer_quality`, `compressed_quality`, `thumbnail_quality`
`media.video.*` (video only)	`proxy_large`, `proxy_medium`, `stream` (HLS), `thumbnail`, `sprite_small`, `sprite_full`
`GET /media/{id}/formats/download`	`original`, `video_large`, `video_medium` (photo: `original`, `compressed`)

```
GET /media/130000990/formats/download   → pick video_medium (720p) or video_large (1080p)
GET <file_url>                          → the proxy; the URL lasts 24 hours
```

No editing and no compute from ScorePlay for the Hackathon.
Cut, crop and transcode on your own machine (ffmpeg, handout).

Twelve Labs finds the moment, ScorePlay keeps it

ScorePlay

the media library

your app

Twelve Labs

video understanding

Twelve Labs	What it gives you
Marengo (search)	a query in words or an image returns clips: `video_id`, `rank`, `start`, `end` in seconds
Pegasus (analyze)	text from a video: summaries, answers, JSON to a schema you define

a Marengo clip `start`, `end` (seconds) → a ScorePlay log `start_offset`, `end_offset` (seconds)

Both in seconds from the start of the video.

The main edit, on your machine: cut, stitch, 9:16, overlay

```
# 1 cut a moment from the proxy, frame-accurate
ffmpeg -ss 00:12:31 -to 00:12:46 -i proxy.mp4 -c:v libx264 -c:a aac a.mp4
# 2 stitch clips: list.txt holds  file 'a.mp4'  then  file 'b.mp4'
ffmpeg -f concat -safe 0 -i list.txt -c copy reel.mp4
# 3 reframe to 9:16, 1080x1920
ffmpeg -i reel.mp4 -vf "crop=ih*9/16:ih,scale=1080:1920" -c:a copy reel_9x16.mp4
# 4 overlay a logo, top right (a transparent PNG keeps its transparency)
ffmpeg -i reel_9x16.mp4 -i logo.png -filter_complex "[1]scale=300:-1[l];[0][l]overlay=W-w-40:40" -c:a copy final.mp4
```

- **ffmpeg** is open source and runs on your laptop: github.com/FFmpeg/FFmpeg.
- **Then:** upload `final.mp4` into your sub-collection (slide 24) and log on the source what you used (slide 23).

Write back tags, details and collections

```
POST /media/{id}/tags     {"tag_option_ids": [...], "player_ids": [...]}   → 204
PUT  /media/{id}/details  {"description": "...", "timecode": "HH:MM:SS:FF"}  → 204
POST /event  {"name": "AI Shorts 9:16", "parent_id": 1469761,
              "external_id": "team10-ai-shorts", "tag_options": [{"id": ...}]}
```

Write a log: a start, an end, and your reasoning

No logs on your platform yet: this is what you write.

example

```
POST /video/logs
{"company_id": 5167, "video_logs": [{"media_id": 130000990,
  "log": {"start_offset": 4210.0, "end_offset": 4228.5,
          "comment": "Example log: your Marengo hit goes here",
          "raw_tags": ["pass"], "player_ids": [728708]}}]}
→ true
```

Read it back: watch the units

Field	Unit
`start_offset`, `end_offset`	seconds from the start
`timestamp`	whole seconds
`start_timestamp`, `end_timestamp`	**milliseconds**

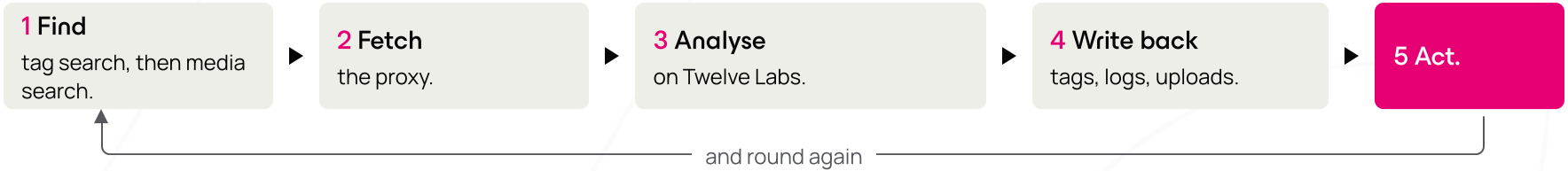
Put content back in three steps

```
POST /upload/event/{collection_id}?parts=1   {"name": "clip_9x16.mp4"}
  → {"upload_id": ..., "key": ..., "urls": [{"url": ...}], "media_id": ...}
PUT <url>   (the file bytes)   → keep the ETag response header
POST /upload/complete/{media_id}
  {"key": ..., "upload_id": ..., "parts_number": 1, "parts": [{"parts_number": 1, "etag": ...}]}
```

Then `status`: `WAITING` › `PROCESSING` › `TRANSFORMED` › `DONE`

About 40 seconds for an 8 second video. The upload then runs through the workflow of slide 5.

The loop, the two challenges, and where to get help



Assisted Auto-Production

one match in, what comes out, and for whom?

Rights and Compliance

what can be published, where, and how do you show why?

Help: ScorePlay and Twelve Labs engineers are on the floor both days. Agree your taxonomy with us before writing. Docs: developer.scoreplay.io (Try-it console, OpenAPI, Postman).

Your toolkit: what to install, where to read

Install	Command
ffmpeg	macOS `brew install ffmpeg`, Ubuntu `sudo apt install ffmpeg`, Windows `winget install Gyan.FFmpeg`
ScorePlay	no SDK needed: any HTTP client, one header

Read	Where
ScorePlay API	developer.scoreplay.io (Try-it console, OpenAPI, Postman)
Twelve Labs	docs.twelvelabs.io: Search, Analyze, Metadata
ffmpeg	source: github.com/FFmpeg/FFmpeg · docs: ffmpeg.org/ffmpeg.html · stitching: trac.ffmpeg.org/wiki/Concatenate